Практическая работа 5. Работа с матрицами

Перед тем, как начать работать с матрицами, скажем несколько слов о типе dim3 – одном из базовых типов CUDA. Он представляет собой не что иное, как вектор из 3-х целочисленных значений. Создается он следующим образом:

```
dim3 v1(512, 2, 4);
dim3 v2(512, 128);
dim3 v3(1024).
```

Если какие-то значения опущены (как в примере с v2 и v3), то они по умолчанию равны 1 (а не 0). То есть вышеприведенный код эквивалентен следующему:

```
dim3 v1(512, 2, 4);
dim3 v2(512, 128, 1);
dim3 v3(1024, 1, 1).
```

В CUDA есть несколько векторных типов для доступа к данным. Например, такой тип, как [u]int может быть 1-, 2-, 3-, 4-мерным вектором. Для создания переменной такого типа применяется функция специального вида, например:

```
int3 a = make_int3 ( 1, 2, 3);
```

Также существуют типы (u)char, (u)short, (u)long, float, которые могут быть 1-, 2-, 3-, 4-мерными векторами, а вот типы long long и double могут быть только 1- и 2-мерными.

На основе типа uint3, в CUDA существует специальный тип dim3, который имеет конструктор. Конструктор этого типа позволяет задавать не все компоненты 3-мерного вектора. Недостающие компоненты этого типа в конструкторе инициализируются единицами. Главное назначение типа dim3 – это задание параметров запуска ядра.

Внутри каждого вычислительного ядра содержатся встроенные переменные, которые используют вышеописанные типы. С помощью этих встроенных переменных можно оперировать отдельными нитями и различать их. Эти переменные были описаны выше, кроме переменной ***warpSize*** типа int – эта переменная содержит значение размера связки нитей (варп), которое на текущий момент всегда равно 32.

Используя встроенные переменные, можно задавать разные параметры запуска ядра и различать отдельные нити в блоках или сами блоки.

Создание матриц

Создадим на CPU матрицу A размером 10x10 и зададим элементы матрицы следующим образом:

$$A = \begin{pmatrix} 0 & 1 & 2 & ... & 9 \\ 10 & 11 & 12 & ... & 19 \\ 20 & 21 & 22 & ... & 29 \\ ... & ... & ... & ... & ... \\ 90 & 91 & 92 & ... & 99 \end{pmatrix}$$

Создадим на GPU матрицу B размером 10x10. Элементы матрицы B зададим такими же, как и у матрицы A. При этом не будем использовать копирование данных с хоста на девайс. Необходимо скопировать матрицу B с девайса на хост и сравнить на хосте две матрицы – A и B. Нужно добиться, чтобы обе матрицы совпадали. Попробуйте запускать программу с разными размерами grid и block.

Для решения этой задачи используем один блок с десятью нитями так, чтобы каждая нить создавала один элемент матрицы. Для удобства блок нитей мы сделаем двумерным. Таким образом, индекс элемента матрицы вычислим как threadIdx.y * n + threadIdx.x. В итоге код ядра будет выглядеть так:

```
__global__ void createMatrix(int *A, const int n)
```

```
{
    A[threadIdx.y * n + threadIdx.x] = 10 *
    threadIdx.y + threadIdx.x;
}
```

На CPU матрицу A будем создавать с помощью двойного цикла следующим образом:

```
size_t size = n * n * sizeof(int);
int *h_A = (int *)malloc(size);
for (int i = 0; i < n; i++)
  for (int j = 0; j < n; j++)
    h_A[j * n + i] = 10 * j + i;
```

Вызов ядра, описанного выше, осуществляется таким образом:

```
dim3 threadsPerBlock = dim3(10, 10);
dim3 blocksPerGrid = dim3(1);

createMatrix<<<blocksPerGrid, threadsPerBlock>>>
(d_B, n);
```

Проверку равенства двух матриц осуществляем следующим образом с помощью двойного цикла:

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
  if (h_A[j * n + i] != h_B[j * n + i]) {
        printf("h_A[%d] != h_B[%d]\n", j * n + i, j
        * n + i);
  }
```

Листинг кода.

```
// Сравнение двух матриц
#include "cuda_runtime.h"
```

```
#include "device_launch_parameters.h"
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
// Ядро
__global__ void createMatrix(int *A, const int n)
{
  // Создание элементов матрицы на GPU
    A[threadIdx.y * n + threadIdx.x] = 10 *
    threadIdx.y + threadIdx.x;
}
int main()
{
  // кол-во строк и столбцов матрицы
    const int n = 10;

    // размер матрицы
    size_t size = n * n * sizeof(int);

    // выделяем память для матрицы на CPU
    int *h_A = (int *)malloc(size);

    // инициализируем матрицу
    for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
    h_A[j * n + i] = 10 * j + i;
    int *d_B = NULL;
    // выделяем память для матрицы на GPU
    cudaMalloc((void **)&d_B, size);

    // определение размеров сетки и блоков
    dim3 threadsPerBlock = dim3(10, 10);
    dim3 blocksPerGrid = dim3(1);

    // вызов ядра
    createMatrix<<<blocksPerGrid, threadsPerB-
    lock>>>(d_B, n);
```

```
    // выделяем память для матрицы B, чтобы
    // скопировать из GPU на CPU
    int *h_B = (int *)malloc(size);

    // копируем матрицу из GPU на CPU
    cudaMemcpy(h_B, d_B, size, cudaMemcpyDeviceTo-
    Host);

    // проверяем совпадение матрицы A и матрицы B
    for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
            if (h_A[j * n + i] != h_B[j * n + i]) {
                printf("h_A[%d] != h_B[%d]\n", j * n
                + i, j * n + i);
            }

    // освобождаем память на GPU
    cudaFree(d_B);

    // освобождаем память на CPU
    free(h_A);
    free(h_B);
    return 0;
}
```

Транспонирование матрицы

Для решения этой задачи используем блоки размером 16x16. Каждая нить блока создает один элемент транспонированной матрицы. Код ядра будет выглядеть так:

```
__global__ void matrixTranspose (const BASE_TYPE *A,
BASE_TYPE *AT, int rows, int cols)
{
    // Индекс элемента в исходной матрице
```

```
    int iA = cols * (blockDim.y * blockIdx.y +
    threadIdx.y) + blockDim.x * blockIdx.x +
    threadIdx.x;
    // Индекс соответствующего элемента в
    // транспонированной матрице
    int iAT = rows * (blockDim.x * blockIdx.x +
    threadIdx.x) + blockDim.y * blockIdx.y +
    threadIdx.y;  AT[iAT] = A[iA];
}
```

Для удобства вычислений размеры исходной матрицы делаем кратными размеру блока. Для этого используем функцию toMultiple(int a, int b), которая возвращает число, кратное b и наиболее близкое к a. Матрицу инициализируем случайными числами.

```
// Инициализация матрицы
for (int i = 0; i < rows * cols; ++i)
{
    h_A[i] = rand()/(BASE_TYPE)RAND_MAX;
}
```

Вызов ядра, описанного выше, осуществляется таким образом:

```
dim3 threadsPerBlock = dim3(BLOCK_SIZE, BLOCK_SIZE);

dim3 blocksPerGrid = dim3(cols / BLOCK_SIZE, rows /
BLOCK_SIZE);

//вызов ядра
matrixTranspose<<<blocksPerGrid, threadsPerB-
lock>>>(d_A, d_AT, rows, cols);
```

Осуществляем проверку правильности работы ядра.

```
for (int i = 0; i < rows; i++)
```

```
  for (int j = 0; j < cols; j++)
        if (h_A[i * cols + j] != h_AT[j * rows + i])
        {
            fprintf(stderr, "Result verification
            failed at element [%d, %d]!\n", i, j);
            exit(EXIT_FAILURE);
    }
```

Листинг кода:

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>
#include <stdlib.h>

// размер блока
#define BLOCK_SIZE 16

// тип, который будут иметь элементы матриц
#define BASE_TYPE float

// Ядро
// Функция транспонирования матрицы
__global__ void matrixTranspose(const BASE_TYPE *A,
BASE_TYPE *AT, int rows, int cols)
{
    // Индекс элемента в исходной матрице
    int iA = cols * (blockDim.y * blockIdx.y +
    threadIdx.y) + blockDim.x * blockIdx.x +
    threadIdx.x;

    // Индекс соответствующего элемента в
    // транспонированной матрице
    int iAT = rows * (blockDim.x * blockIdx.x +
    threadIdx.x) + blockDim.y * blockIdx.y +
    threadIdx.y;
    AT[iAT] = A[iA];
}
```

```
// Функция вычисления числа, которое больше числа a
// и кратное числу b
int toMultiple(int a, int b)
{
    int mod = a % b;
    if (mod != 0)
    {
        mod = b - mod;
        return a + mod;
    }
    return a;
}

int main()
{
    // Объекты событий
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

  // Количество строк и столбцов матрицы
    int rows = 1000;
    int cols = 2000;

  // Меняем количество строк и столбцов матрицы
    // на число, кратное размеру блока (16)
    rows = toMultiple(rows, BLOCK_SIZE);
    printf("rows = %d\n", rows);
    cols = toMultiple(cols, BLOCK_SIZE);
    printf("cols = %d\n", cols);
    size_t size = rows * cols * sizeof(BASE_TYPE);

  // Выделение памяти под матрицы на хосте
  // Исходная матрица
    BASE_TYPE *h_A = (BASE_TYPE *)malloc(size);
  // Транспонированная матрица
    BASE_TYPE *h_AT = (BASE_TYPE *)malloc(size);
```

```
// Инициализация матрицы
  for (int i = 0; i < rows * cols; ++i)
  {
      h_A[i] = rand()/(BASE_TYPE)RAND_MAX;
  }
// Выделение глобальной памяти на девайсе
  // для исходной матрицы
  BASE_TYPE *d_A = NULL;
  cudaMalloc((void **)&d_A, size);

  // Выделение глобальной памяти на девайсе для
  // транспонированной матрицы
  BASE_TYPE *d_AT = NULL;
  cudaMalloc((void **)&d_AT, size);

  // Копируем матрицу из CPU на GPU
  cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDe-
  vice);

  // Определяем размер блока и сетки
  dim3 threadsPerBlock = dim3(BLOCK_SIZE,
  BLOCK_SIZE);
  dim3 blocksPerGrid = dim3(cols / BLOCK_SIZE,
  rows / BLOCK_SIZE);

  // Начать отсчета времени
  cudaEventRecord( start, 0);

  // Запуск ядра
  matrixTranspose<<<blocksPerGrid, threadsPerB-
  lock>>>(d_A, d_AT, rows, cols);

  // Окончание работы ядра, остановка времени
  cudaEventRecord( stop, 0);
  cudaEventSynchronize( stop );
  float KernelTime;
  cudaEventElapsedTime( &KernelTime, start, stop);
```

```
    printf("KernelTime: %.2f milliseconds\n", Ker-
    nelTime);

    // Копируем матрицу из GPU на CPU
    cudaMemcpy(h_AT, d_AT, size, cudaMemcpyDeviceTo-
    Host);

    // Проверка правильности работы ядра
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
        {
            if (h_A[i * cols + j] != h_AT[j * rows +
            i])
            fprintf(stderr, "Result verification
            failed at element [%d, %d]!\n", i, j);
            exit(EXIT_FAILURE);
        }
    printf("Test PASSED\n");

    // Освобождаем память на GPU
    cudaFree(d_A);
    // Освобождаем память на GPU
    cudaFree(d_AT);
  // Освобождаем память на CPU
    free(h_A);
    free(h_AT);
      // Удаляем объекты событий
    cudaEventDestroy( start );
  cudaEventDestroy( stop );
  printf("Done\n");
  return 0;
}
```

Сложение матриц

Поскольку сложение матриц выполняется поэлементно (еще проще, чем умножение), то все сказанное в следующем разделе (по поводу параллельного

вычисления и представления массивов в памяти) справедливо и для сложения матриц.

Для решения задачи используем функцию __global__ void matrixAdd.

```
// Ядро
__global__ void matrixAdd(const BASE_TYPE *A, const
BASE_TYPE *B, BASE_TYPE *C, int cols)
{
    // Вычисление индекса элемента матрицы на GPU
    int ind = cols * (blockDim.y * blockIdx.y +
    threadIdx.y) + blockDim.x * blockIdx.x + threadIdx.x;

    C[ind] = A[ind] + B[ind];
}

// Функция вычисление числа, которое больше
// числа a и кратное числу b
int toMultiple(int a, int b)
{
     int mod = a % b;
     if (mod != 0)
     {
     mod = b - mod;
     return a + mod;
     }
     return a;
}
```

Умножение матриц

Пусть даны матрицы $A = (a_{ij})$ и $B = (b_{ij})$ размерами $m \times n$ и $n \times p$ соответственно. Тогда произведение $C = (c_{ij}) = AB$ размерностью $m \times p$, как известно, определяется по формуле

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}.$$

Элементы матрицы-произведения зависят только от элементов исходных матриц, поэтому их вычисление может выполняться параллельно, без дополнительных «ухищрений». Поэтому рассмотрим лишь вопрос о вариантах представления матрицы в памяти – в виде двухмерного (например, float** a, затем обращение вида a[i][j]) или одномерного массива (float* a и затем обращение вида a[i * N + j], где N – длина строки (количество столбцов)).

Рассмотрим вопрос о том, как работает операция индексирования:

1) в случае с одномерным массивом:

a) вычисляется выражение i * N + j (в современных CPU это выполняется за одну операцию, так называемую fused multiply-add);

b) полученное число (смещение) умножается на размер типа и складывается с базовым адресом (адресом начала массива);

c) как правило, переменные i, j и базовый адрес будут храниться в регистрах CPU, поэтому вышеприведенные операции выполняются быстро.

2) в случае с двухмерным массивом:

a) значение i умножается на размер указателя и складывается с базовым адресом, вычисляется адрес a[i] (начала i-й строки);

b) Значение j умножается на размер элемента и складывается с адресом a[i], вычисляется адрес a[i][j];

c) Адрес a[i], очевидно, не хранится в регистре, поэтому операция с ним будет медленнее.

С другой стороны, одномерный массив требует большого *последовательного* участка памяти.

Для решения задачи опишем функцию __global__ void matrixMult.

```
__global__ void matrixMult(const BASE_TYPE *A, const
BASE_TYPE *B, BASE_TYPE *C, int Acols, int Bcols)
{
    int i0 = Acols * (blockDim.y * blockIdx.y +
    threadIdx.y);
```

```
    int j0 = blockDim.x * blockIdx.x + threadIdx.x;
    BASE_TYPE sum = 0;

    for (int k = 0; k < Acols; k++)
      sum += A[i0 + k] * B[k * Bcols + j0];

    int ind = Bcols * (blockDim.y * blockIdx.y +
    threadIdx.y) + blockDim.x * blockIdx.x + threadIdx.x;

    C[ind] = sum;
}
```

Листинг кода:

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>
#include <stdlib.h>

#define BLOCK_SIZE 16
// тип, который будут иметь элементы матриц
#define BASE_TYPE double
// функция перемножения матриц
__global__ void matrixMult(const BASE_TYPE *A, const
BASE_TYPE *B, BASE_TYPE *C, int Acols, int Bcols)
{
    int i0 = Acols * (blockDim.y * blockIdx.y +
    threadIdx.y);
    int j0 = blockDim.x * blockIdx.x + threadIdx.x;
    BASE_TYPE sum = 0;

    for (int k = 0; k < Acols; k++)
      sum += A[i0 + k] * B[k * Bcols + j0];

    int ind = Bcols * (blockDim.y * blockIdx.y +
    threadIdx.y) + blockDim.x * blockIdx.x + threadIdx.x;

    C[ind] = sum;
}
```

```
int toMultiple(int a, int b) {

  int mod = a % b;

  if (mod != 0) {
    mod = b - mod;
    return a + mod;
  }
  return a;
}

int main()
{
  //start, stop - for Kernel time
    cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);

    // количество строк и столбцов матрицы
    int Arows = 100;
    int Acols = 200;
    int Brows = Acols;
    int Bcols = 150;

    Arows = toMultiple(Arows, BLOCK_SIZE);
    printf("Arows = %d\n", Arows);

    Acols = toMultiple(Acols, BLOCK_SIZE);
    printf("Acols = %d\n", Acols);

    Brows = toMultiple(Brows, BLOCK_SIZE);
    printf("Brows = %d\n", Brows);

    Bcols = toMultiple(Bcols, BLOCK_SIZE);
    printf("Bcols = %d\n", Bcols);

    size_t Asize = Arows * Acols * sizeof(BASE_TYPE);
    size_t Bsize = Brows * Bcols * sizeof(BASE_TYPE);
    size_t Csize = Arows * Bcols * sizeof(BASE_TYPE);
```

```
  BASE_TYPE *h_A = (BASE_TYPE *)malloc(Asize);
  BASE_TYPE *h_B = (BASE_TYPE *)malloc(Bsize);
  BASE_TYPE *h_C = (BASE_TYPE *)malloc(Csize);

  for (int i = 0; i < Arows * Acols; ++i) {
      h_A[i] = rand()/(BASE_TYPE)RAND_MAX;
  }
  for (int i = 0; i < Brows * Bcols; ++i) {
      h_B[i] = rand()/(BASE_TYPE)RAND_MAX;
  }

  BASE_TYPE *d_A = NULL;
  cudaMalloc((void **)&d_A, Asize);

  BASE_TYPE *d_B = NULL;
  cudaMalloc((void **)&d_B, Bsize);

  BASE_TYPE * d_C = NULL;
  cudaMalloc((void **)&d_C, Csize);
  cudaMemcpy(d_A, h_A, Asize, cudaMemcpyHostToDevice);

  cudaMemcpy(d_B, h_B, Bsize, cudaMemcpyHostToDevice);

  dim3 threadsPerBlock = dim3(BLOCK_SIZE, BLOCK_SIZE);
  dim3 blocksPerGrid = dim3(Bcols / BLOCK_SIZE, Arows /
  BLOCK_SIZE);

  cudaEventRecord( start, 0);

  matrixMult<<<blocksPerGrid, threadsPerBlock>>>(d_A,
  d_B, d_C, Acols, Bcols);

  cudaEventRecord( stop, 0);
cudaEventSynchronize( stop );
float KernelTime;
cudaEventElapsedTime( &KernelTime, start, stop);
  printf("KernelTime: %.2f milliseconds\n", Ker-
  nelTime);
```

```
    cudaMemcpy(h_C, d_C, Csize, cudaMemcpyDeviceToHost);

    printf("Test STARTED\n");
    for (int i = 0; i < Arows; i++) {
    for (int j = 0; j < Bcols; j++) {
        BASE_TYPE sum = 0;
        for (int k = 0; k < Acols; k++)
                sum += h_A[i * Acols + k] * h_B[k *
                Bcols + j];

            if (fabs(h_C[i * Bcols + j] - sum) > 1e-3)
            {
                fprintf(stderr, "Result verification
                failed at element [%d, %d]!\n", i, j);
                printf("sum = %f, h_C[i * Bcols + j] =
                %f\n", sum, h_C[i * Bcols + j]);
                 exit(EXIT_FAILURE);
            }
          }
    }
    printf("Test PASSED\n");

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    free(h_A);
    free(h_B);
    free(h_C);
    cudaEventDestroy( start );
  cudaEventDestroy( stop );

  return 0;
}
```

Практическая работа

1. Проверьте, является ли данная квадратная матрица ортогональной, путем умножения на транспонированную (не выделяя под нее дополнительной памяти) и сравнения результата с единичной матрицей.
2. Проверьте, являются ли две заданные квадратные матрицы A и B коммутирующими (т.е. их произведение не зависит от порядка умножения: AB = BA).
3. Напишите функцию для сложения двух матриц M×N, заданных как вектор векторов (float a[M][N], b[M][N]). Матрицы задаются на хосте, после ко-пируются на девайс, и там производится сложение.

Контрольные вопросы

1. Чем отличается создание матрицы на GPU от создания матрицы на CPU и последующего копирования ее элементов на GPU?
2. В каких участках кода при операциях над матрицами возникают потери в производительности?